DIO: Hybrid Cost Routing, Session Affinity, and Sound Admission for Multi-Instance LLM Serving over Stock vLLM

Keyush Nisar[1*], Krishil Parikh[1] and Krisha Maisheri[1]

[1*]Department of Computer Engineering, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India.

*Corresponding author(s). E-mail(s): Nisarkeyush3@gmail.com; Contributing authors: Krishilparikh75@gmail.com; KrishaMaisheri16@gmail.com;

Abstract

Multiple stock vLLM (or OpenAI-compatible) instances are often placed behind Round-Robin or least-connections balancers. Effective service cost still diverges across peers—queue depth, KV-cache pressure, prefix locality, temporary slowdowns—even when GPU SKUs match. Absolute latency predictors trained on black-box completion telemetry are also unreliable: dual Tesla T4 runs show NLMS MAPE of ∼90–130%, so admission that gates on predicted milliseconds is unsafe.

DIO (`dio-serve`) is a non-invasive OpenAI-compatible gateway with three mechanisms. (A) **Hybrid cost routing** ranks with dual-timescale NLMS (request e2e latency and tokens) and corrects scores with Prometheus signals scraped from each stock vLLM `/metrics` endpoint (KV-cache utilization, waiting-queue depth, prefix-cache hit rate), without engine source patches. (B) **Session/prefix affinity** steers multi-turn and agentic traffic toward the replica already holding useful prefix state and reports affinity hit rate as an explicit metric. (C) **Sound admission** separates ranking from reject/admit: `rank_only` and `empirical` (observed-percentile) modes avoid the false rejects of absolute-$\hat{y}$ gates under high MAPE.

On dual-T4 + Qwen2.5-3B + stock vLLM ($\boldsymbol{n}$**=10** seeds), homogeneous NLMS does not beat Round-Robin on mean p99; under controlled **×2** service-time skew, NLMS cuts p99 by **48.3% ± 0.7%** vs. RR. Separate *library* multi-seed suites (not GPU wall-clock) isolate hybrid steering, affinity stickiness, and admission disagreement. Code and artifacts are public. Multi-SKU fleets are out of scope.

Keywords: LLM serving, multi-instance load balancing, vLLM, hybrid routing, prefix-cache affinity, admission control, NLMS, Prometheus metrics.

1 Introduction

1.1 Motivation

Practitioners commonly run *multiple* stock vLLM [1] (or SGLang/TGI [2–4]) instances and put Nginx [5], Kubernetes Services [6], Envoy [7], or Ray Serve [8] in front with Round-Robin or least-connections. Single-node engines optimize continuous batching [9, 10]; they do not solve *which backend* should take the next request. Classic L7 balancers [5, 11] treat backends as equal capacity units.

In production, effective cost still diverges even on identical GPU SKUs: pending queues grow unevenly, KV-cache utilization spikes on one replica, prefix caches favor sessions pinned to a peer, and host interference can temporarily slow one process. The result is avoidable tail latency [12]. Co-designed cluster schedulers [13–15] can react with deep engine hooks or live KV migration, but many operators need a *non-invasive* front door that wraps stock OpenAI-compatible [16] engines without patches.

A second problem is quieter but equally important: online latency models learned from black-box e2e telemetry are often accurate only as *relative rankers*. On dual Tesla T4 we measure dual-timescale NLMS [17] MAPE of ∼90–130%. Systems that still gate admission on "reject if $\min \hat{y} > \text{SLO}$" inherit that error and can starve healthy traffic.

1.2 Approach

We design **DIO** (`dio-serve`) as a thin control plane for the multi-instance pattern (Fig. 1). The closed loop uses coarse request telemetry (e2e latency, tokens, gateway pending) *and*, when available, stock engine metrics already exported on each vLLM Prometheus [18] endpoint—still HTTP-only, still zero source modification. Three mechanisms work together:

(A) **Hybrid cost routing.** NLMS provides an online ranking signal from completions; scraped KV usage, waiting requests, and prefix hit rate supply ground-truth corrections that close part of the MAPE/telemetry gap without sacrificing engine-agnosticism (Section 4).

(B) **Session/prefix affinity.** Multi-turn and agentic workloads [19, 20] benefit when follow-ups land on the replica holding prefix state [21–23]. DIO makes affinity a first-class cost term and metric, not a minor bonus (Section 5).

(C) **Sound admission.** Ranking always uses $\arg\min S_w$; admission is a separate policy—`rank_only`, `empirical`, or diagnostic `absolute`—so unreliable absolute $\hat{y}$ cannot thrash reject/admit [24, 25] (Section 6).

1.3 Results preview

Dual-timescale NLMS does **not** invent wins when backends are equal: on homogeneous dual-T4, NLMS p99 is within a few percent of Round-Robin and slightly worse (Section 7.3). Where NLMS helps is service-time skew: under controlled ×2 e2e inflation on one real peer, NLMS reduces p99 by $48.3\% \pm 0.7\%$ vs. RR and beats d=2 RLS

(Section 7.6). Hybrid metrics, affinity, and admission are evaluated as full mechanisms in Sections 7.7–**??**.

1.4 Contributions

1. **Hybrid cost routing (non-invasive).** Formal joint cost that fuses online NLMS with scraped vLLM `/metrics`; graceful degrade when scrape fails.
2. **Session/prefix affinity for multi-turn routing.** Prefix-hash stickiness plus engine prefix-hit bonus, with multi-seed stickiness and latency vs. RR (library suite; dual-T4 multi-turn re-measure future).
3. **Sound admission under unreliable $\hat{y}$.** Explicit rank-only / empirical / absolute modes; quantitative disagreement under poisoned predictors (library suite, motivated by dual-T4 MAPE).
4. **Open gateway and multi-seed evaluation.** Open-source `dio-serve`; dual-T4 + Qwen2.5-3B [26]; no free lunch on homogeneous peers; multi-SKU fleets out of scope.

2 Background and related work

2.1 Single-node engines and prefix/KV systems

vLLM [1] popularized PagedAttention and continuous batching; alternatives revisit memory management without paging [27]. SGLang [2, 4] improves structured decoding and RadixAttention prefix reuse. TGI [3] targets production APIs. Hybrid iteration policies include Sarathi-Serve [10] and FastServe [28]. Disaggregated prefill/decode systems [25, 29–31] optimize pipeline stages and KV placement. Prefix/KV reuse [21–23, 32, 33] raises the value of *where* a request lands. These systems excel on one node or a co-designed cluster; multi-worker routing in front of *stock* engines remains external.

2.2 Cluster orchestrators and classical serving

Ray Serve [8] and Triton [34] emphasize actors and model repositories with coarse metrics. Earlier DNN serving stacks (Clipper [35], INFaaS [36], Clockwork [24]) established predictability and model-selection themes that LLM gateways inherit. Orca [9] advances iteration-level batching. AlpaServe [37] and CoCoServe [38] focus on placement and multiplexing. Mélange [39] targets heterogeneous capacity tables. NexusSched [13] combines predictive routing with engine co-design and multi-feature predictors ($O(d^2)$ updates). Llumnix [14] migrates live KV state. LoongServe [15] targets long-context elastic parallelism. ServerlessLLM [40] optimizes cold-start loading. DIO complements these as a *non-invasive request-level front end*: dual-timescale scalar NLMS, hybrid Prometheus scrape [18], joint cost, and stock engines only—no engine patches and no live KV migration.

2.3 Adaptive prediction, queues, and admission

NLMS stability and step-size practice follow adaptive filter theory [17, 41]; recursive least squares is the natural $O(d^2)$ comparator. Queue wait uses a batch-amortized

Little-style heuristic [42, 43]. SJF-style ranking motivates predicted service routing [44]. Roofline [45] inspires soft memory ceilings recast as routing penalties. Goodput- and SLO-aware LLM serving [25, 46, 47] motivates treating admission as a first-class policy. Simulation frameworks such as Vidur [48] characterize offline capacity; DIO targets online multi-instance ranking under weak absolute models.

3 System design

3.1 Architecture

DIO separates a **control plane** from a **data plane** of unmodified engines (Fig. 1). Clients send OpenAI-compatible `/v1/chat/completions` (or `/v1/completions`) to DIO. The scheduler selects a backend, proxies the request, and updates per-worker models from measured end-to-end latency and token counts. Product path: `dio-serve` (FastAPI + Python scheduler). Optional research path: Go manager with gRPC workers.

Telemetry contract.

The closed loop uses four channels: (i) wall-clock e2e latency through the gateway; (ii) token counts from response `usage` or a tokenizer estimate; (iii) per-backend `pending` depth inside DIO; (iv) optional hybrid scrape of each backend's `/metrics`. Soft/hard VRAM terms can be config-driven or inferred from KV usage. We do **not** claim continuous NVML SM/power/temperature telemetry, and we do not patch engine source—in contrast to co-designed control planes that require deep hooks [13, 14].

3.2 Dual-timescale NLMS latency model

For worker w and approximate token count N, we model

$$\hat{y}_w = s_w N + b_w, \tag{1}$$

where s_w is ms/token and b_w is fixed overhead. On completion with residual $e = y - \hat{y}$, NLMS updates

$$s \leftarrow s + \mu \frac{e}{N}, \qquad b \leftarrow b + \mu_b e, \tag{2}$$

with dual rates μ_{fast}=0.1, μ_{slow}=0.01, μ_b=0.005, and

$$s^{\text{eff}} = \alpha s^{\text{fast}} + (1 - \alpha) s^{\text{slow}}, \quad \alpha{=}0.8. \tag{3}$$

Cold-start priors are s_0=2.0 ms/token, b_0=150 ms. We distinguish (i) $O(1)$ arithmetic per update from (ii) time-to-usable ranking: typically **15–20 completions per worker** before slopes leave priors. Figure 2 illustrates dual-timescale response to an injected slope jump in simulation. Token feature N prefers Hugging Face tokenizer counts plus planned `max_tokens`; fallback is $\lfloor |\text{prompt}|/4 \rfloor +$ `max_tokens`.

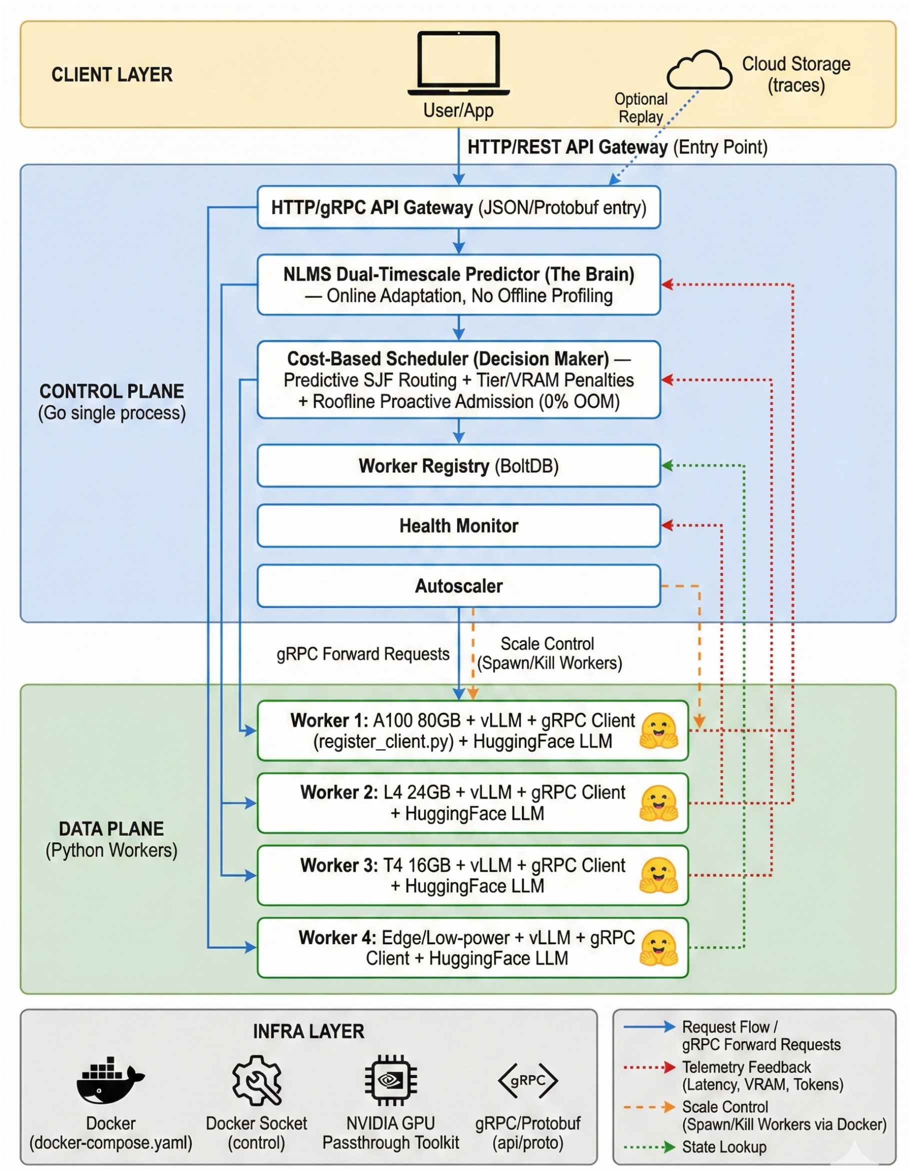

Fig. 1 DIO multi-instance gateway. Clients talk OpenAI HTTP to DIO. The control plane ranks backends with dual-timescale NLMS, hybrid Prometheus scrape from each stock vLLM, session affinity, and decoupled admission. Data-plane engines are unmodified.

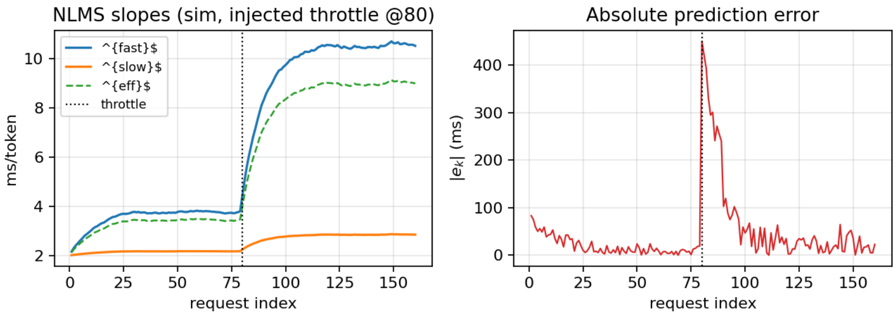

Fig. 2 NLMS dual-timescale adaptation under an injected true-slope jump at request 80 (simulation). s^{fast} reacts first; s^{slow} lags. Synthetic illustration, not dual-T4 wall-clock traces.

4 Hybrid cost routing with engine metrics

4.1 Problem: black-box ranking misses engine ground truth

NLMS observes only gateway e2e latency and tokens. That signal is delayed, noisy, and confounded by batching. Meanwhile stock vLLM already exports internal state via Prometheus [18]: GPU KV-cache utilization, number of waiting and running requests, and prefix-cache hit counters. Prior multi-instance balancers either ignore these signals or require invasive co-design [13, 14]. DIO's hybrid design is deliberately coarse: **scrape what the engine already publishes**, fold it into the same millisecond-scale cost used for ranking, and keep zero source modification.

4.2 Metric scrape and snapshot

For each backend i with base URL U_i and metrics path (default `/metrics`), a background task issues `GET` U_i`/metrics` and parses Prometheus exposition text. Metric name variants across vLLM versions are handled (e.g. `vllm:gpu_cache_usage_perc` vs. `vllm:kv_cache_usage_perc`; prefix hit rate from a direct gauge or from hits/queries counters). Each scrape yields a snapshot

$$\mathcal{E}_w = (\mathrm{KV}_w,\, W_w,\, R_w,\, \mathrm{Hit}_w,\, \mathrm{ok}_w), \tag{5}$$

where $\mathrm{KV}_w \in [0,1]$, W_w and R_w are waiting/running request counts, $\mathrm{Hit}_w \in [0,1]$ is prefix-cache hit rate, and ok_w is false on HTTP/parse failure. On failure, hybrid terms contribute zero (graceful degrade to NLMS-only ranking). Scrape interval defaults to 1 s—fast enough to track KV pressure, slow enough not to contend with decode.

4.3 Hybrid joint cost

When ok_w is true, the full score is

$$\begin{aligned} S_w = {} & \mathrm{wait}_w + \hat{t}_w + \mathrm{tierCost}_{r,w} + \mathrm{vramCost}_w \\ & + c_{\mathrm{kv}}\, \mathrm{KV}_w + c_q\, W_w - \mathrm{cacheBonus}_w - c_p\, \mathrm{Hit}_w. \end{aligned} \tag{6}$$

Default hybrid coefficients (same ms units as other terms): c_{kv}=800, c_q=50 per waiting request, c_p=150. These were chosen so that a near-full KV cache (KV≈1) contributes on the order of one long-request decode budget, a moderate waiting queue (W≈10) is comparable to a soft VRAM penalty, and a high engine prefix hit rate is of similar scale to the session affinity bonus C=200 ms—i.e., heuristic parity with existing base cost terms rather than a formal grid search. Base tier/cache/VRAM coefficients in prior L4 pilots showed ±50% relative insensitivity under the workshop suite; we did *not* re-grid the hybrid triple on dual-T4 and treat rigorous coefficient tuning as future work. Operators may override all three via `DIO_*` configuration.

Additionally, if KV_w>0.95 and N>500, worker w is hard-blocked for that request (engine-reported near-full KV). Free-VRAM estimates may be updated as $\mathrm{free}_w \approx \mathrm{total}_w(1 - \mathrm{KV}_w)$ when totals are known.

What hybrid is not.

Hybrid cost is *not* a claim of SM utilization, power, or temperature awareness. It is also not a substitute for NLMS: when backends differ only in learned service slope and metrics look similar, NLMS ranking still dominates. The novelty is the combination—lightweight online ranker + non-invasive engine ground truth—still deployable as a stock gateway.

Algorithm 1 Hybrid engine scrape (background)

Require: backends B, interval Δ

loop
    for all backend $b \in B$ **do**
        $T \leftarrow$ HTTP GET b.metrics_url with timeout
        if T ok **then**
            $\mathcal{E}_b \leftarrow \mathrm{ParsePrometheus}(T)$
            $\mathrm{Scheduler.SetEngineMetrics}(b, \mathcal{E}_b)$
        end if
    end for
    sleep Δ
end loop

Algorithm 2 DIO worker selection (hybrid + affinity + admission)

Require: request r, workers W, admission mode m, snapshots $\{\mathcal{E}_w\}$

$N \leftarrow \mathrm{TokenFeature}(r)$; $w^\star \leftarrow \perp$; $S_{\min} \leftarrow \infty$
for all $w \in W$ healthy and tier-feasible **do**
    $\hat{t}_w \leftarrow \mathrm{Predict}(w, N)$
    if VRAM/KV hard-blocked$(w, N, \mathcal{E}_w)$ **then continue**
    end if
    $S_w \leftarrow \mathrm{wait} + \hat{t}_w + \mathrm{tier} + \mathrm{vram} + c_{\mathrm{kv}}\mathrm{KV}_w + c_q W_w$
    $-\mathrm{cacheBonus}(r, w) - c_p \mathrm{Hit}_w$
    if $S_w < S_{\min}$ **then** $S_{\min} \leftarrow S_w$; $w^\star \leftarrow w$
    end if
end for
if $w^\star = \perp$ **or** $\mathrm{AdmissionReject}(S_{\min}, m)$ **then**
    return HTTP 503 with Retry-After
end if
$\mathrm{RecordPrefixAffinity}(r, w^\star)$; **return** $w^\star$

5 Session and prefix-cache affinity

5.1 Problem: multi-turn locality is underserved by RR

Round-Robin and least-connections scatter multi-turn sessions across replicas. Each turn that lands on a cold replica re-prefills shared system prompts and agent context, missing in-engine prefix caches [4, 21, 22]. Cache-aware and prompt-scheduling systems [23, 32] show large gains when locality is preserved, but they often assume co-designed clusters. DIO targets the common case: several stock replicas behind one OpenAI gateway, multi-turn or agentic clients [19, 20], no engine patches.

5.2 Affinity mechanism

On each admitted request with prompt text p, DIO computes a stable prefix hash over the first 256 characters:

$$h(p) = \mathrm{hash}(p[0{:}256]) \bmod 2^{32}. \tag{7}$$

A map $M : h \mapsto w$ records the last worker that successfully served that prefix. When scoring worker w, if $M[h(p)] = w$, a cache affinity bonus $\mathrm{cacheBonus}_w = C$ (default $C{=}200\,\mathrm{ms}$; raised in multi-turn experiments) is subtracted from S_w. Independently, the hybrid term $c_p \mathrm{Hit}_w$ prefers workers the *engine* reports as currently caching well. After admission, DIO sets $M[h(p)] \leftarrow w^\star$.

Metrics.

DIO maintains affinity_decisions and affinity_hits (true when the chosen worker already owned the prefix). Exposed hit rate affinity_hits/affinity_decisions is the primary stickiness metric for multi-turn evaluation. When live vLLM metrics are available, operators can also compare engine-reported prefix hit rates under DIO vs. RR.

Relationship to in-engine caching.

Affinity is request-level stickiness complementary to RadixAttention [4]: DIO increases the chance that the engine's own cache is warm; it does not implement a second cache. Ablating `no_cache` disables the bonus for sensitivity studies.

6 Sound admission under unreliable latency prediction

6.1 Problem: absolute gates trust a weak model

Many predictive schedulers implicitly assume $\hat{y}$ is calibrated enough to compare against an SLO in absolute milliseconds. Section 7.4 shows that assumption fails for NLMS on dual-T4 (MAPE $\sim$90–130%). Cold-start priors and batching can make S_w larger than any reasonable SLO even when true latency is fine. Gating on $\min_w S_w > \mathrm{SLO}$ then rejects healthy traffic—or, if priors are optimistic, admits work that will miss the SLO. Predictability-first serving [24] and goodput-oriented LLM systems [25, 46] motivate treating admission as a policy that must not inherit model bias.

6.2 Three admission modes

Ranking is always $\arg\min_w S_w$ among feasible workers. Admission is separate:

`rank_only`
: Never reject for SLO based on S_w. Only hard VRAM/tier/KV blocks apply. NLMS is used purely for ranking. Preferred for routing A/B tests and when the operator handles overload externally.

`empirical` (default)
: Maintain a rolling window of observed e2e latencies (default length 64). After warm-up (≥ 8 samples), compute the p-th percentile (default p=95). Reject only if that empirical tail exceeds the SLO *and* the best current score lies in the worst quartile of available worker scores. Cold-start never rejects on $\hat{y}$ alone.

`absolute` (legacy / diagnostic)
: Reject if $\min_w S_w >$ SLO. Sensitive to MAPE and cold-start scale. Retained so operators can quantify how often absolute would disagree with the active mode.

Algorithm 3 AdmissionReject($S_{\min}, m$)

Require: best score $S_{\min}$, mode m, SLO L, recent e2e window $\mathcal{Y}$

if $m =$ `rank_only` **then return** false
end if
if $m =$ `empirical` **then**
  if $|\mathcal{Y}| < 8$ **then return** false ▷ warm-up: do not trust $\hat{y}$
  end if
  $q \leftarrow$ Percentile$(\mathcal{Y}, p)$
  $Q_{75} \leftarrow$ Percentile of current feasible scores at 75%
  return $(q > L) \wedge (S_{\min} \geq Q_{75})$
end if
return $(S_{\min} > L)$ ▷ `absolute`

Diagnostics.

Counters `would_reject_absolute`, `would_admit_absolute`, and `absolute_vs_active_disagree` record, on every pick, whether absolute mode would have made a different decision. These are the primary paper metrics for admission safety (Section **??**). Rejected requests return HTTP 503 with `Retry-After`.

7 Experimental evaluation

7.1 Methodology

Hardware and software (real GPU).

Dual NVIDIA Tesla T4, stock vLLM [1], Qwen2.5-3B-Instruct [26], Hugging Face tokenizer, `dio-serve` gateway. Workshop suite: `run_workshop_final_suite.py`.

Regimes (non-interchangeable).

1. **Regime A — homogeneous dual-T4 (real):** identical peers, `max_tokens`=32, n=10 seeds × 30 requests.
2. **Regime B — legacy Locust pilot (secondary):** longer-decode ShareGPT [49]/arXiv/Azure-style traces, single seed; absolute seconds not mixed with Regime A ms.
3. **Regime C — service-time skew:** (i) CPU multi-seed simulation n=10; (ii) real dual-T4 with e1 behind delay-proxy ×2, n=10.
4. **A/B/C mechanism suites (library multi-seed, *not* GPU):** hybrid pressure, multi-turn affinity, admission under poisoned $\hat{y}$; n=10 (`run_abc_experiments.py`). Synthetic service times and injected metrics; isolate mechanism correctness complementary to dual-T4 cells (Sections 7.7–**??**).

Baselines and metrics.

Round-Robin (RR), Least-Loaded (LL), RLS [17] (d=2, λ=0.99) under the same engines where applicable. Primary metrics: p50/p99 e2e mean±std; MAPE; fraction to backend e0; affinity hit rate / stickiness; admission reject rate, completions, absolute-vs-active disagreement. We do not claim a matched multi-node Orca [9] or vLLM-distributed bake-off.

7.2 Control-plane overhead

On a scalability microbench with many concurrent mock workers, instrumented scheduling completes in approximately **14 μs** per request (worker scan, cost evaluation, prefix hash). Per-worker state is ∼256 B. The control plane is not the bottleneck relative to LLM decode (Fig. 3).

7.3 Regime A: homogeneous dual-T4 (no free lunch)

Table 1 is the primary homogeneous multi-seed matrix. On *identical* backends, **NLMS does not beat Round-Robin on mean p99** (slightly worse: −2.6% ± 1.0%). This is the correct outcome if ranking is driven by learned service cost and workers truly look alike. Secondary signals: NLMS keeps tight p99 std (31 ms) while Least-Loaded is sticky (100% to e0, p99 std 430 ms); RLS mis-routes (frac e0 0.12).

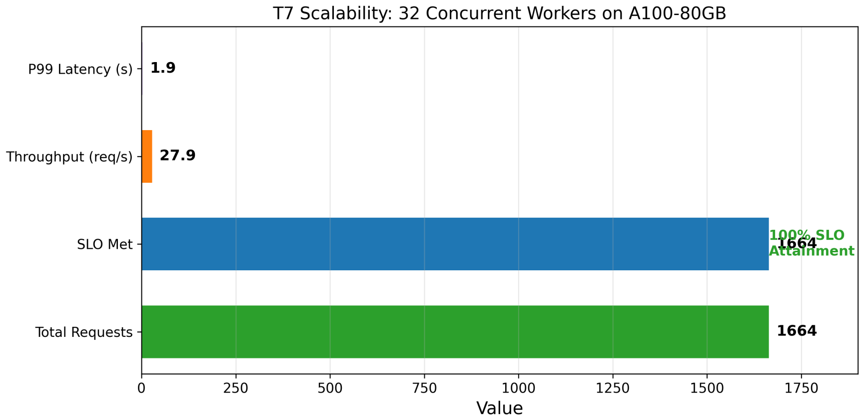

Fig. 3 Control-plane scalability microbench. Scheduling overhead remains $\sim 14\,\mu$s; plotted e2e latency is synthetic worker delay.

Table 1 Regime A (real, primary). Dual-T4 + vLLM + Qwen2.5-3B, HF tokenizer, n=10 × 30, `max_tokens`=32. Mean±std.

Strategy	p99 (ms)	MAPE (%)	frac e0	vs RR
NLMS	3413 ± 31	123 ± 6	0.45 ± 0.04	$-2.6 \pm 1.0\%$
RLS (d=2)	3466 ± 11	132 ± 6	0.12 ± 0.02	$-4.2 \pm 0.6\%$
Round-Robin	3326 ± 17	88 ± 2	0.50 ± 0.00	—
Least-Loaded	3313 ± 430	141 ± 13	1.00 ± 0.00	$+0.4 \pm 13\%$

7.4 MAPE versus relative ranking

NLMS MAPE on dual-T4 is **high** ($123 \pm 6\%$ with tokenizer). HF tokenizer vs. byte heuristic (W2, n=3): MAPE 117.5±10.9% vs. 89.7±4.9%—tokenizer is **worse**, so error is structural (linear model, cold start, batching), not mere token counting. Routing uses $\arg\min S_w$; ranking under skew is the success criterion (Regime C). Absolute admission that trusts $\hat{y}$ is therefore unsafe (Section **??**).

7.5 Regime B: legacy Locust pilot (secondary)

Table 2 retains a single-seed Locust pilot (longer decode, four logical workers). Absolute seconds are **not** comparable to Regime A milliseconds; we report it only for continuity with earlier pilots.

7.6 Regime C: service-time skew

Simulation** (n=10)**.

With distinct true (b, s) pairs, NLMS places 97.5% of traffic on the fast worker and cuts p99 by $38.3\% \pm 2.0\%$ vs. RR (Table 3). RLS fails to rebalance (frac fast 0.40 ± 0.18). Local `pick()` overhead: NLMS $\approx 9.6\,\mu$s, RLS $\approx 9.4\,\mu$s at d=2.

Table 2 Regime B (legacy single-seed Locust pilot). Not multi-seed; different setup from Table 1.

Trace	Strat.	p99 (s)	RPS	Fail %
ShareGPT	NLMS	**67**	**0.37**	0
ShareGPT	RR	81	0.29	0
arXiv	NLMS	52	**0.36**	0
arXiv	RR	51	0.30	0
Azure	NLMS	48	0.33	0
Azure	RR	47	0.36	0

Table 3 Regime C (simulation). Slope-skew dual workers, n=10 × 200.

Metric	NLMS	RR	Notes
Frac. to fast	0.975 ± 0.000	0.500 ± 0.000	
p99 (sim units)	1411 ± 34	2287 ± 49	
p99 vs RR	$38.3\% \pm 2.0\%$		

Real dual-T4 service-time skew (n=10).

We inflate e1 wall-clock e2e by ×2 via `latency_delay_proxy` while still serving real tokens on a real T4—a controlled *slow peer*, not a second GPU SKU. Table 4: NLMS p99 3427 ± 38 ms vs. RR 6632 ± 37 ms ($48.3\% \pm 0.7\%$ improvement). RLS is weaker and noisier ($23.2\% \pm 22.9\%$). Mean frac to e0 under NLMS is modest (0.47 ± 0.08): the supported claim is **better p99 under skew than RR/RLS**, not sim-style 97.5% sticky splits.

Note on NLMS p99 near Regime A.

NLMS p99 under Regime C (3427 ± 38 ms) is numerically close to Regime A (3413 ± 31 ms). This is expected, not a table copy error: both cells use the same dual-T4 + Qwen2.5-3B + HF-tokenizer probe workload and seed count; under NLMS the router largely avoids the delayed peer, so the admitted path's wall-clock tail resembles the homogeneous unthrottled case. RR under Regime C is much worse (6632 ms) because half the traffic hits the ×2 peer.

Table 4 Regime C (real GPU, primary). Dual-T4 + delay-proxy ×2, n=10 × 30, HF tokenizer. NLMS p99 is intentionally near Table 1 because NLMS avoids the slow peer (see note above).

Strategy	p99 (ms)	frac e0	vs RR	MAPE (%)
NLMS	$\mathbf{3427 \pm 38}$	0.47 ± 0.08	$\mathbf{48.3 \pm 0.7}$%	126 ± 12
RLS (d=2)	5087 ± 1505	0.37 ± 0.19	$23.2 \pm 22.9\%$	119 ± 10
Round-Robin	6632 ± 37	0.50 ± 0.00	—	102 ± 3

Longer decode.

Homogeneous dual-T4, `max_tokens`=128, n=3 × 20: NLMS 13835 ± 73 ms vs. RR 13821 ± 88 ms (tied), matching Regime A.

7.7 Hybrid cost: mechanism correctness (library multi-seed, not GPU)

Scope (read this first).

The results in this subsection are from the **library multi-seed suite** `run_abc_experiments.py`: synthetic service times and *injected* engine snapshots, no vLLM process and no GPU wall-clock. They establish mechanism correctness and multi-seed statistics for hybrid scoring, and are **complementary to**—not a substitute for—the real dual-T4 cells in Sections 7.3–7.6.

Setup.

n=10 seeds × 120 requests. Two workers share the same true service law $y = 2N + 100 + \epsilon$. One worker is labeled *hot* via injected snapshots (high KV, high W) and accrues extra queue delay when pending builds; the other is *cool*. We compare Hybrid (NLMS + hybrid terms), NLMS-only (no engine metrics), and Round-Robin.

Results.

Figure 4 and Table 5: hybrid and NLMS-only both avoid the hot worker once queue delay appears in e2e feedback (frac cool 1.00), while RR stays balanced (0.50). Hybrid p99 is 257 ± 3 vs. RR 292 ± 6 (12.0% ± 2.0%). NLMS-only reaches a similar p99 after feedback reveals the hot queue; hybrid adds earlier, direct use of KV/queue ground truth and hard KV blocks under near-full cache. On a live cluster the scrape period (∼1 s) is still short relative to decode, so the same terms can act as a continuous correction while NLMS adapts slopes—that live re-measurement is left for future dual-T4 cells.

Table 5 Library suite (not GPU). Hybrid cost under injected engine pressure, n=10 × 120. Mean±std. Complements dual-T4 Tables 1–4.

Mode	frac cool	p99 (sim ms)	p99 vs RR
Hybrid (NLMS+metrics)	1.00 ± 0.00	257 ± 3	**12.0** ± **2.0**%
NLMS-only	1.00 ± 0.00	258 ± 5	∼12%
Round-Robin	0.50 ± 0.00	292 ± 6	—

Takeaway.

Hybrid cost is the non-invasive answer to "MAPE is high": do not invent better absolute ms from black-box telemetry alone; fuse ranking with engine-exported KV and queue signals that stock vLLM already exposes [1, 18].

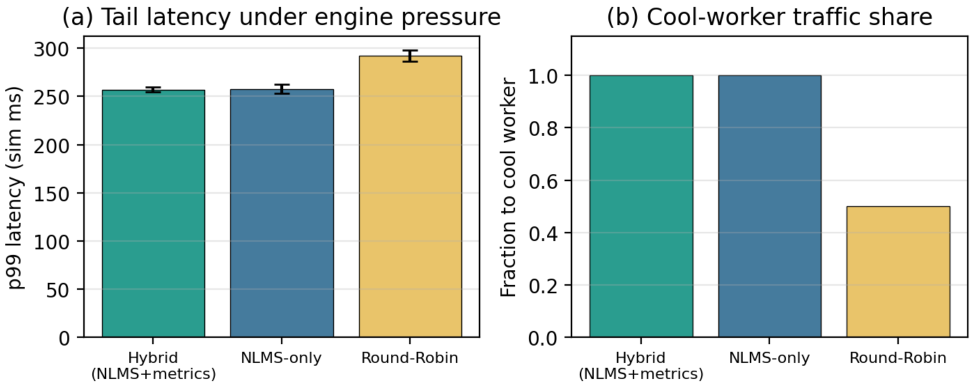

Fig. 4 Library suite (not GPU). Hybrid cost under injected engine pressure, n=10 × 120. (a) p99 (sim ms); (b) fraction to the cool worker.

Table 6 Library suite (not GPU). Session/prefix affinity multi-turn suite, n=10. Mean±std. Complementary to dual-T4 regimes.

Mode	Affinity / proxy hit	Stickiness	p99 (sim ms)
NLMS+affinity	0.80 ± 0.00	1.00 ± 0.00	190 ± 4
Round-Robin	0.00 ± 0.00	0.60 ± 0.00	221 ± 3

Table 7 Library suite (not GPU). Admission modes under poisoned absolute $\hat{y}$ (true $y <$ SLO, n=10 × 120). Mean±std.

Mode	Reject rate	Completed	Goodput	Abs. disagreements
`absolute`	1.00 ± 0.00	0 ± 0	0.00	0
`empirical`	0.00 ± 0.00	120 ± 0	1.00	120 ± 0
`rank_only`	0.00 ± 0.00	120 ± 0	1.00	120 ± 0

7.10 Ablations and resilience

On an L4 stressed long-prompt microbench, removing VRAM admission elevates hard failures (~23%); removing queue wait raises tail latency; removing tiers hurts mixed-capability mixes (Fig. 7). Cold-start and overload behaviors are summarized in Fig. 8: under intentional overload, DIO returns HTTP 503 rather than unbounded queueing—consistent with empirical/rank-only safety rather than absolute-$\hat{y}$ thrashing.

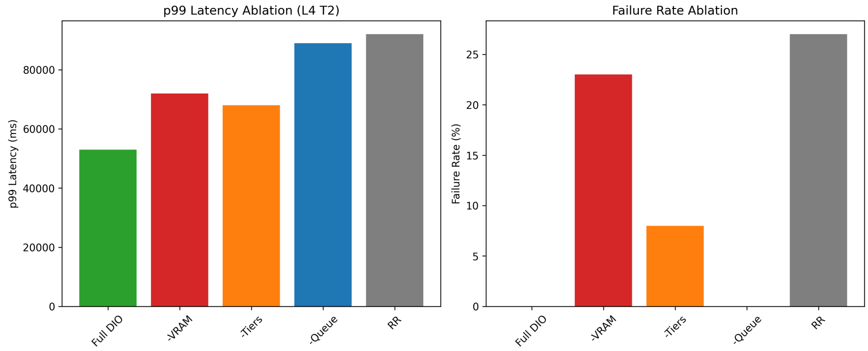

Fig. 7 Cost-term ablation (L4 microbench). Full DIO keeps failures low and tail controlled.

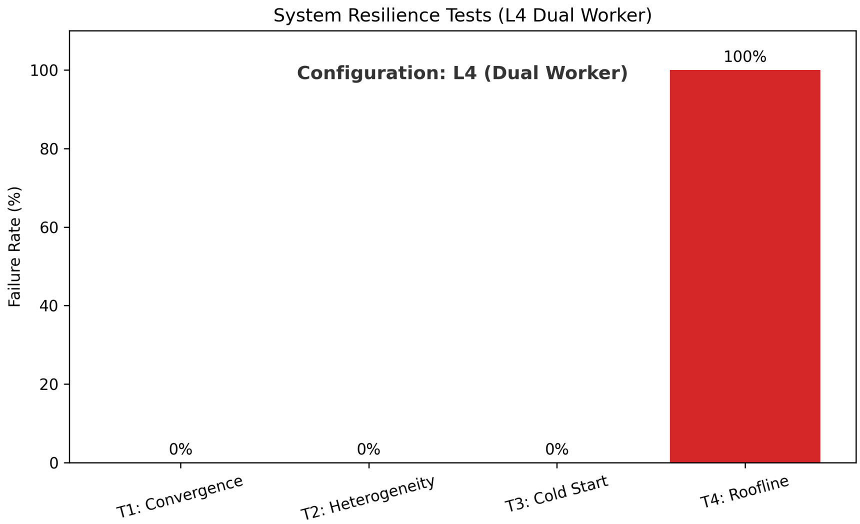

Fig. 8 Resilience microbenchmarks (L4 suite). Overload exercises admission (503), not throughput maximization.

8 Discussion and limitations

When each mechanism matters.

NLMS ranking alone helps under observed service skew (Section 7.6), not when peers match (Section 7.3). Hybrid scrape terms matter when KV or waiting-queue pressure is visible before e2e residuals catch up (Section 7.7; library suite). Affinity matters for multi-turn locality on multi-replica single-SKU fleets (Section ??). Prefer `empirical` or `rank_only` admission whenever MAPE is high enough that absolute S_w is not a safe SLO proxy (Section ??). In practice: enable hybrid scrape when `/metrics` exists, keep affinity on for chat/agent traffic, and do not gate production traffic on absolute-$\hat{y}$ alone.

Limitations.

(i) Hybrid telemetry is limited to stock vLLM Prometheus exports (no continuous NVML SM/power loop). (ii) Real skew is delay-proxy on identical T4s, not multi-SKU hardware (explicitly out of scope). (iii) Hybrid, affinity, and admission multi-seed tables above are library mechanism suites; concurrent multi-turn dual-T4 re-measurement of live prefix-cache hit rate remains future work. (iv) Baselines are RR/LL/RLS on the same vLLM pair, not multi-node Orca/vLLM-distributed; dual-T4 multi-seed cells are mostly short sequential probes rather than concurrent ShareGPT.

9 Conclusion

DIO (`dio-serve`) is a non-invasive multi-instance gateway for stock vLLM-class engines with three mechanisms: hybrid cost routing (NLMS plus scraped KV/queue/prefix metrics), session/prefix affinity for multi-turn traffic, and admission that does not gate on absolute $\hat{y}$ when MAPE is high. Dual-T4 multi-seed results show no free lunch on matched peers and a large p99 gain under controlled service-time skew. Library suites establish hybrid, affinity, and admission mechanism correctness. Code and artifacts are public.

Acknowledgements. The authors thank colleagues who provided feedback on earlier drafts.

Declarations

- **Funding.** No external funding was received for this work.
- **Conflict of interest/Competing interests.** The authors declare that they have no competing interests.
- **Ethics approval and consent to participate.** Not applicable (no human subjects or personal data).
- **Consent for publication.** Not applicable.

- **Data availability.** Aggregated experimental results are included in the repository under `results_workshop_final/`, `results_abc/`, and related directories. Scripts regenerate library and GPU suites.
- **Materials availability.** Not applicable.
- **Code availability.** Source code for `dio-serve` (hybrid `/metrics` scrape, affinity, admission modes), validation suites (`run_workshop_final_suite.py`, `run_abc_experiments.py`, delay-proxy harness), and paper artifacts are available at https://github.com/nisaral/DIO.
- **Author contribution.** K.N. led system design, implementation, experiments, and manuscript drafting. K.P. and K.M. contributed to evaluation, analysis, and manuscript revision. All authors approved the final manuscript.

Appendix A Notation summary

Table A1 Main symbols.

Symbol	Meaning
$s^{\text{fast}}, s^{\text{slow}}, s^{\text{eff}}$	Dual-timescale slopes (ms/token)
b_w	Intercept (ms)
$N, y, \hat{y}$	Tokens; actual / predicted e2e latency (ms)
S_w	Joint routing cost for worker w
Q_w, B	Pending depth; batch amortization
$\mathrm{KV}_w, W_w, \mathrm{Hit}_w$	Scraped KV usage, waiting count, prefix hit rate
c_{kv}, c_q, c_p	Hybrid cost coefficients (ms scale)
C	Prefix affinity bonus (ms)
L	Latency SLO (ms)

Appendix B Default coefficients

Default production coefficients: μ_{fast}=0.1, μ_{slow}=0.01, μ_b=0.005, α=0.8, B=8, c_{kv}=800, c_q=50, c_p=150, C=200, VRAM soft/hard 4096/2400 MB, metrics interval 1 s, empirical percentile p=95, recent window 64. Hybrid triple selection is heuristic parity with base terms (Section 4); not dual-T4 grid-searched. All are overridable via `DIO_*` settings.

References

[1] Kwon, W., *et al.*: Efficient memory management for large language model serving with PagedAttention. In: SOSP (2023)

[2] Zheng, L., et al.: SGLang: Efficient Execution of Structured Language Model Programs (2023)

[3] Hugging Face: Text Generation Inference. https://github.com/huggingface/text-generation-inference (2023)

[4] Zheng, L., *et al.*: SGLang: Efficient execution of structured language model programs. In: NeurIPS (2024). RadixAttention prefix cache

[5] F5, Inc.: NGINX HTTP Load Balancer. https://nginx.org/ (2024)

[6] Cloud Native Computing Foundation: Kubernetes. https://kubernetes.io/ (2024)

[7] Envoy Project Authors: Envoy Proxy. https://www.envoyproxy.io/ (2024)

[8] Ray Project: Ray Serve. https://docs.ray.io/en/latest/serve/ (2024)

[9] Yu, G.-I., *et al.*: Orca: A distributed serving system for Transformer-based generative models. In: OSDI (2022)

[10] Agrawal, A., *et al.*: Sarathi-Serve: Efficiently serving large language models via hybrid scheduling. In: OSDI (2024)

[11] Eisenbud, D.E., *et al.*: Maglev: A fast and reliable software network load balancer. In: NSDI (2016)

[12] Dean, J., Barroso, L.A.: The tail at scale. Communications of the ACM **56**(2) (2013)

[13] Zhang, Y., Chen, Y., Mo, X., Xi, A., Li, J., Wu, W.: A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving (NexusSched) (2025)

[14] Wu, B., et al.: Llumnix: Dynamic Scheduling for Large Language Model Serving (2024)

[15] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., Jin, X.: LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism. In: SOSP (2024)

[16] OpenAI: OpenAI API Reference. https://platform.openai.com/docs/api-reference (2024)

[17] Haykin, S.: Adaptive Filter Theory, 4th edn. Prentice Hall, ??? (2002)

[18] Prometheus Authors: Prometheus: Monitoring System and Time Series Database. https://prometheus.io/ (2024)

[19] Wu, Q., et al.: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation (2023)

[20] LangChain: LangGraph: Build Resilient Language Agents as Graphs. https://

github.com/langchain-ai/langgraph (2024)

[21] Gim, I., et al.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference (2023)

[22] Juravsky, J., et al.: Hydragen: High-Throughput LLM Inference with Shared Prefixes (2024)

[23] Srivatsa, V., et al.: Preble: Efficient Distributed Prompt Scheduling for LLM Serving (2024)

[24] Gujarati, A., *et al.*: Serving DNNs like clockwork: Performance predictability from the bottom up. In: OSDI (2020)

[25] Zhong, Y., *et al.*: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: OSDI (2024)

[26] Yang, A., et al.: Qwen2.5 Technical Report (2024)

[27] Prabhu, R., et al.: vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention (2024)

[28] Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., *et al.*: Fast distributed inference serving for large language models. In: arXiv (2023). arXiv:2305.05920

[29] Patel, P., et al.: Splitwise: Efficient Generative LLM Inference Using Phase Splitting. arXiv preprint (2024)

[30] et al.: Mooncake: Trading between Memory and Storage for Efficient LLM Serving. arXiv preprint (2024)

[31] et al.: MemServe: Context Caching for Disaggregated LLM Serving. arXiv preprint (2024)

[32] Yao, J., et al.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion (2024)

[33] Cheng, Y., et al.: LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv / open-source cache layer (2024)

[34] NVIDIA Corporation: Triton Inference Server (2024)

[35] Crankshaw, D., *et al.*: Clipper: A low-latency online prediction serving system. In: NSDI (2017)

[36] Romero, F., *et al.*: INFaaS: Automated model-less inference serving. In: USENIX ATC (2021)